GEMSTONE In-person Meeting 2026

Terraforming GEMSTONE: a hands-on introduction to Terra

6/22/2026

Set up

1. Create a Terra account
2. Download workshop materials from https://github.com/marcoteix/gemstone-terra-workshop

   ```
   cd $HOME && git clone https://github.com/marcoteix/gemstone-terra-workshop && cd gemstone-terra-workshop
   ```

3. Install **Google Cloud SDK**

Goals

We want to investigate *C. auris* colonizing nursing home residents

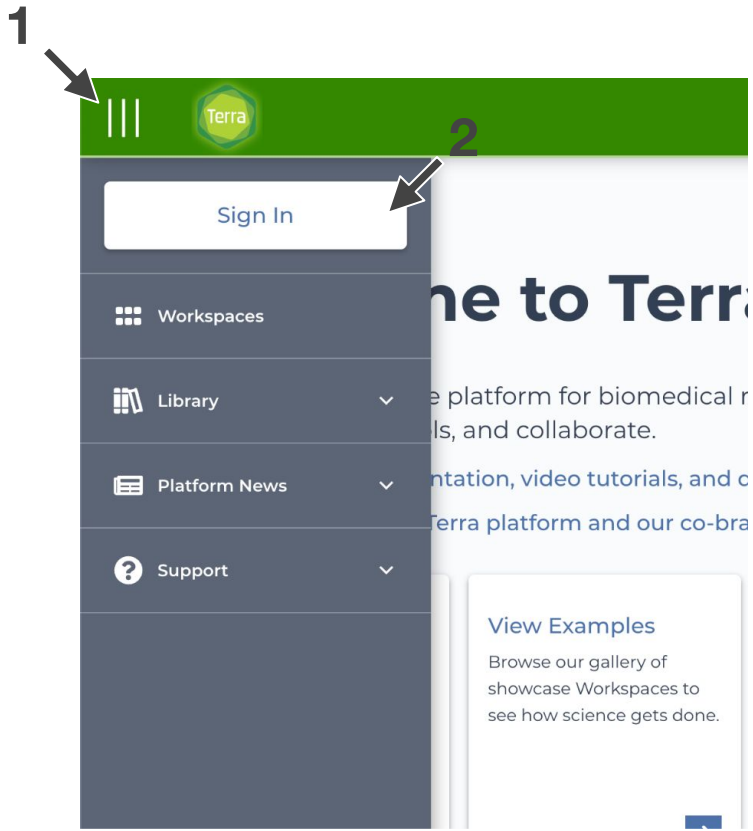
Using Terra, we will:

1. Determine clade types for 10 *C. auris* isolates
2. Infer a phylogenetic tree for clade I isolates
3. Determine which resident characteristics are associated with carrying *C. auris* clade I versus clade III

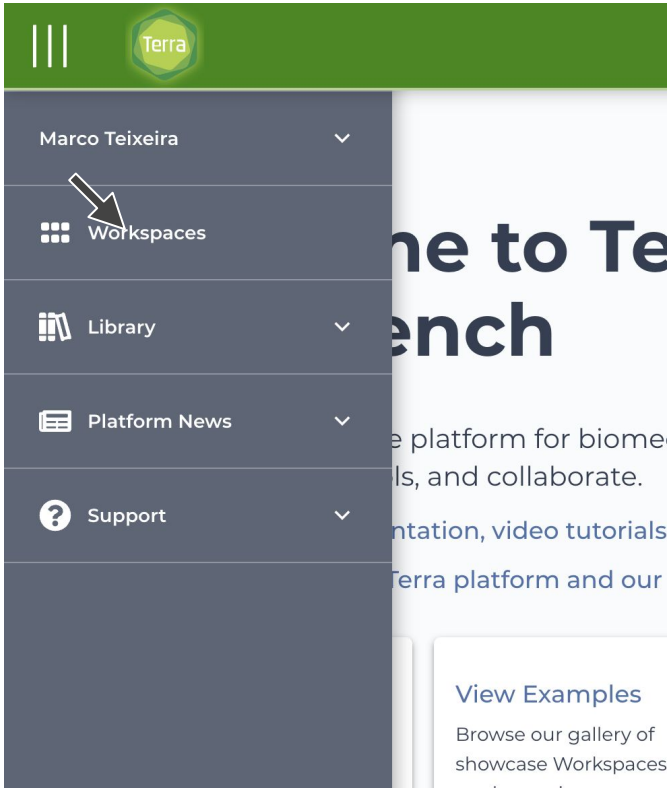
Log into Terra

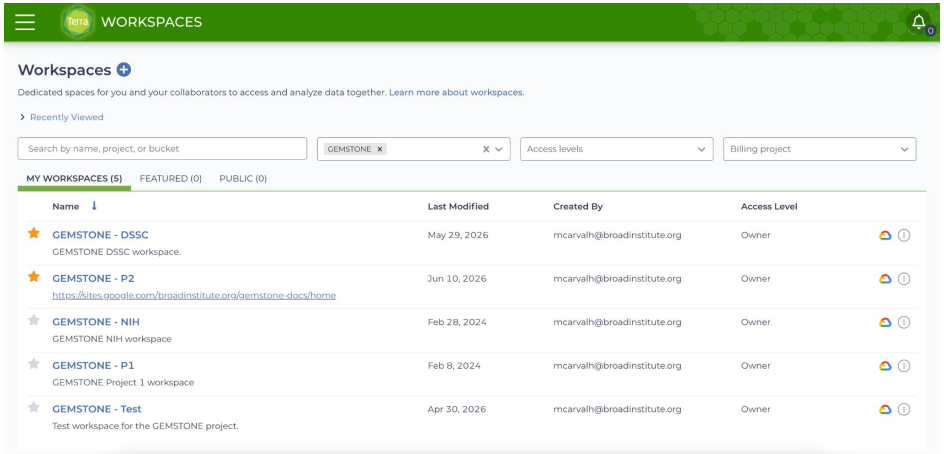
Use your browser of choice to access

https://app.terra.bio/



2 Access your workspaces





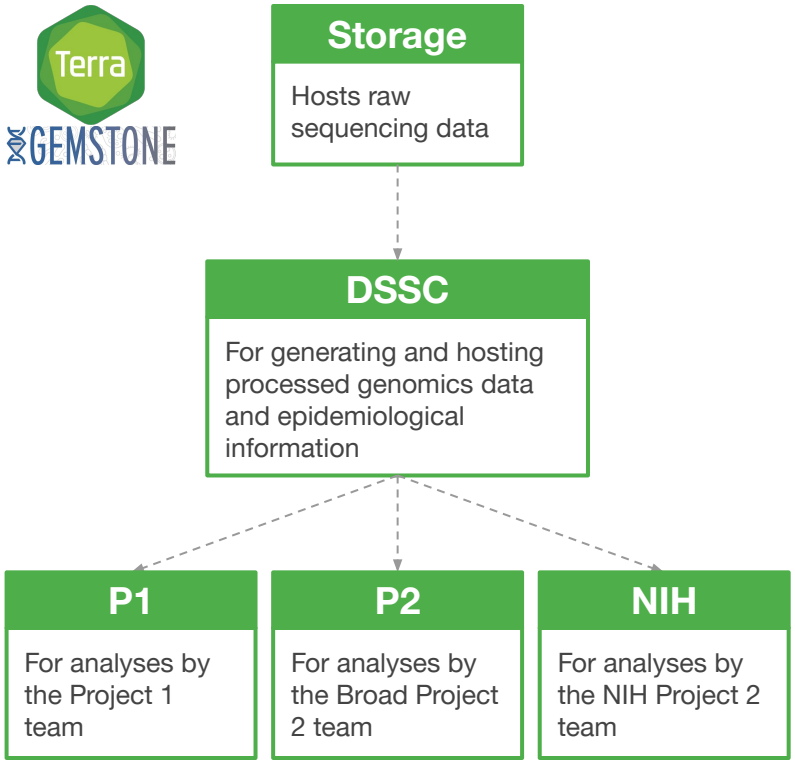
This page lists the workspaces you have access to

GEMSTONE Terra workspaces

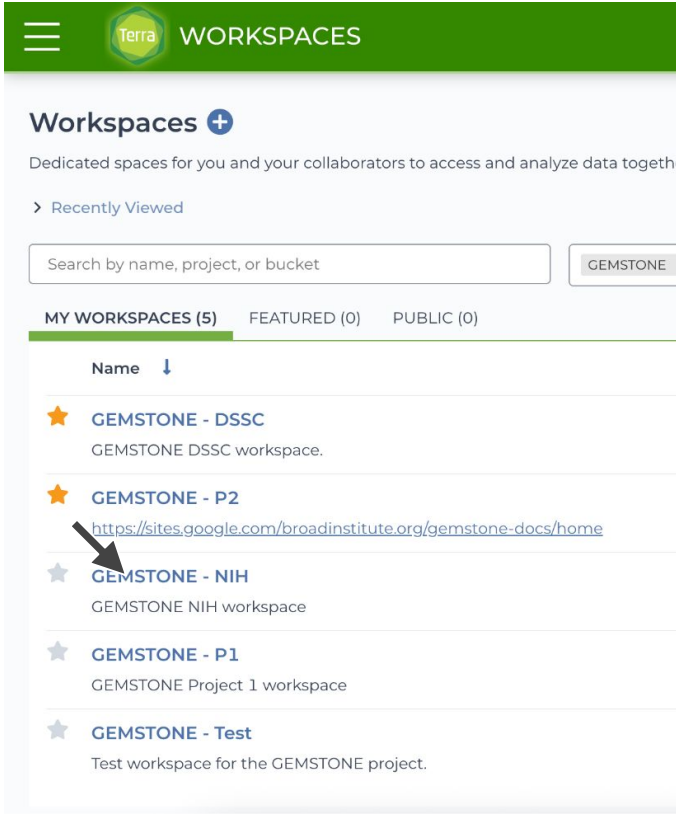
Terra workspaces host data, allow you to perform analyses, and run bioinformatics workflows

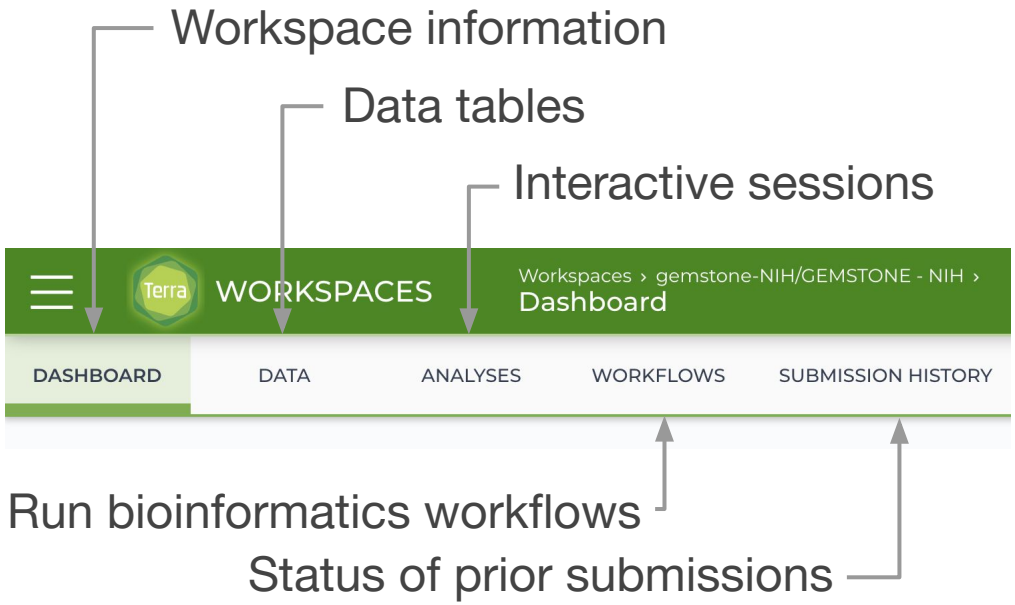
All GEMSTONE Terra users can access data in the **Storage** and **DSSC** workspaces from inside their project's workspace

Each workspace has its own cost object



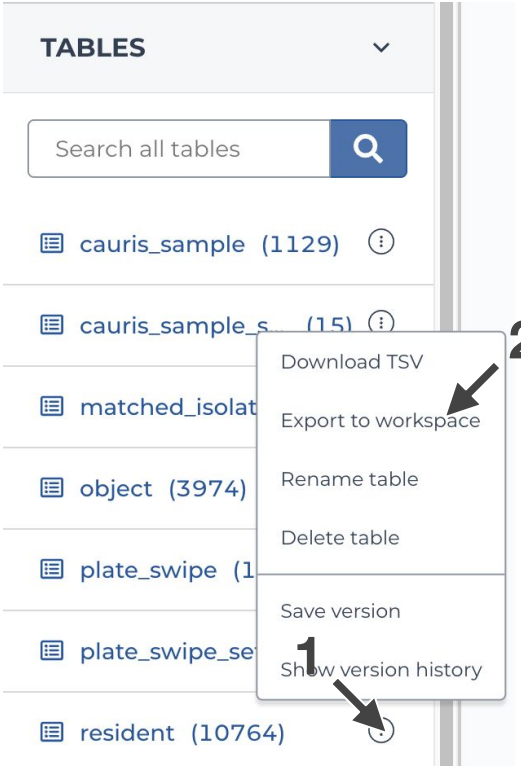
3 Explore your workspace





Copy GEMSTONE data to your workspace

1. Navigate to the GEMSTONE - DSSC workspace
2. Click on the **Data** tab
3. Select the *resident* table and click **Export to workspace**
4. Select your team's workspace
5. Click **copy**
6. Click **Go To Copied Data**



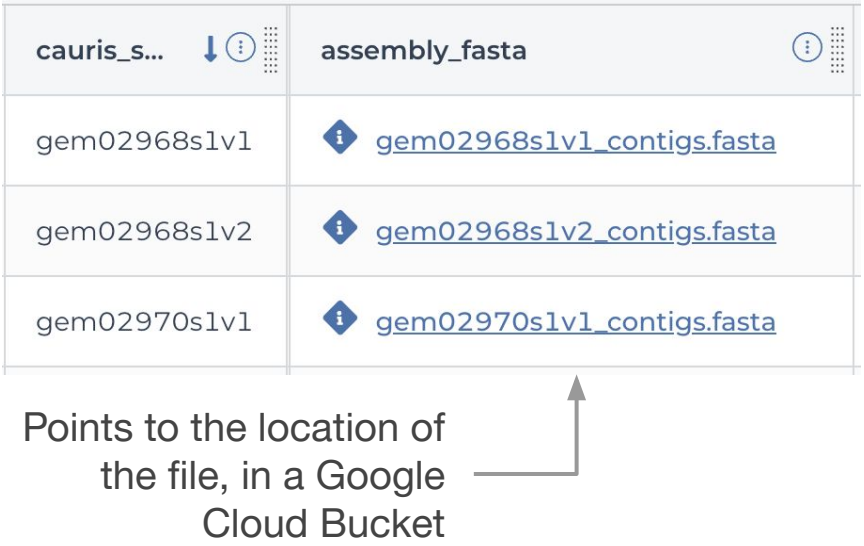
Accessing data on Terra

Data tables contain the location of files generated by bioinformatics tools

These files can be downloaded

- From the GUI, by clicking on the filename and selecting **Download**
- From the command line, using the **Google Cloud SDK**

```
gcloud storage cp \
    $LOCATION $DESTINATION
```



Upload your own data table to Terra

1. Inspect the TSV file in `data/cauris_sample.tsv`

   ```
   head data/cauris_sample.tsv
   ```

Terra data tables must

- Be a TSV file
- Have a unique identifier as the first column
- The first column should be named **`entity:`**`<table_name>`**`_id`**

5 Upload your own data table to Terra

2. Upload the data table using the GUI

- Click **Import Data**
- Select **Upload TSV**
- Drag or select the resident.tsv file
- Click **Start Import Job**
- Wait for a notification at the top of your screen indicating whether the upload was successful

You should see a **resident** data table in the **Data** tab

Note
The name of the data table is inferred from the name of the first column. Uploading an already existing table will append to it or update fields if the IDs in the TSV match an existing ID in the Terra table.

Adding new team members to your workspace

You can add users to your workspace by adding them to your GEMSTONE Terra group

1. Navigate to the **Groups** page using the menu on the top left corner
2. Click on your team's group (e.g., GEMSTONE_NIH)
3. Click **+ Add Users**
4. Type the new user's email address associated with Terra, select a role, and click **Add Users**

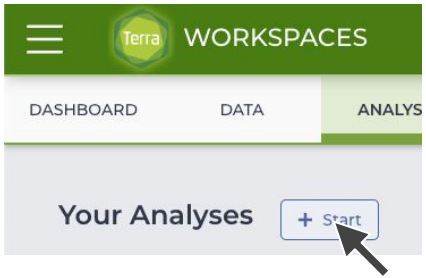
This grants access to:

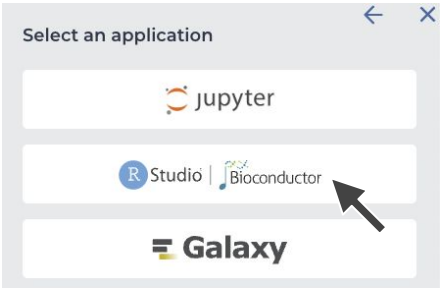
- Your team's workspace (editor)
- GEMSTONE data workspaces (reader)

6 Launch an RStudio interactive session

Terra allows you to do analyses using interactive RStudio, Jupyter, Galaxy, and CLI sessions

1. In your team's workspace, click the **Analyses** tab





2. Name the file *<Your initials>_GEMSTONE_Terra_workshop*
3. Select **R Markdown (.Rmd)** as a file type
4. Click **Create Analysis**
5. Verify the default configuration and click **Create**

Launch an RStudio interactive session

6. Select your analysis file from the **Analyses** tab and wait for the environment to be created

> **Note**
>
> The disk associated with the environment (Persistent Disk) will not be deleted when the environment itself is deleted, unless specified. This incurs costs.
>
> Workspace data is accessible within the environment, but the Persistent Disk cannot be accessed from outside the environment
>
> You should **copy output files you would like to keep onto the workspace Google Bucket**

7. Click 🚀 **Open**

Load data tables into R

1. Rename the file in scripts/gemstone_terra_workshop.Rmd to include your initials

scripts/MT_gemstone_terra_workshop.Rmd

2. Upload the R Markdown file to the CWD and open it in RStudio

3. Run the first two cells.

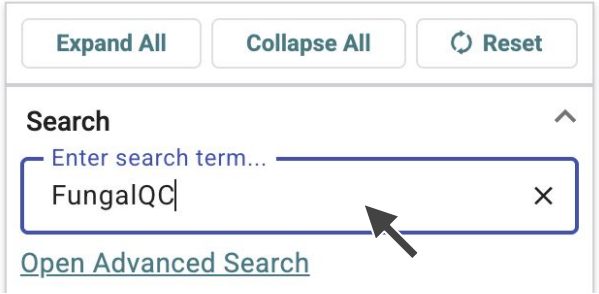
4. Read the **cauris_sample** and **resident** table into R:

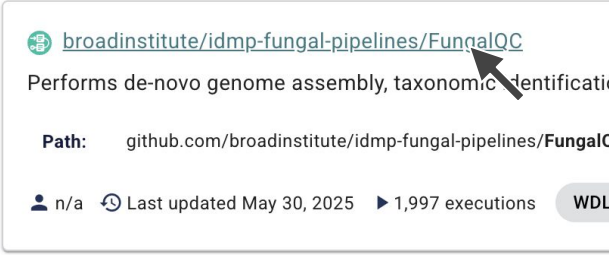
```
# Load C. auris isolate table
cauris <- avtable("cauris_sample")
# Load table with resident metadata
resident <- avtable("resident")
```

5. Inspect the **cauris_sample** table: `head(cauris)`

8 Import the GEMSTONE *C. auris* workflow

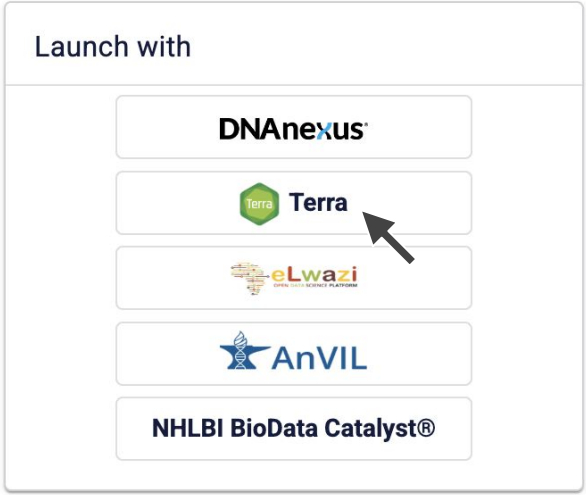
1. Click on the **Workflows** tab
2. Click **Find a Workflow**
3. Click **Dockstore.org**
   Dockstore is a repository of bioinformatics workflows which you can reuse
4. Search for **FungalQC**
   You can find other GEMSTONE workflows by filtering by the *gemstone* tag
5. Select the *broadinstitute/idmp-fungal-pipelines/FungalQC* workflow





8 Import the GEMSTONE *C. auris* workflow

6. Select **Launch with Terra**
7. Select your team's workspace from the dropdown menu
8. Click **Import**
9. Navigate to the **Workflows** tab
10. Select the **FungalQC** workflow



9 Define workflow inputs

Terra workflows operate on data tables in your workspace

- Read inputs from a data table
- Write outputs to columns in the same data table

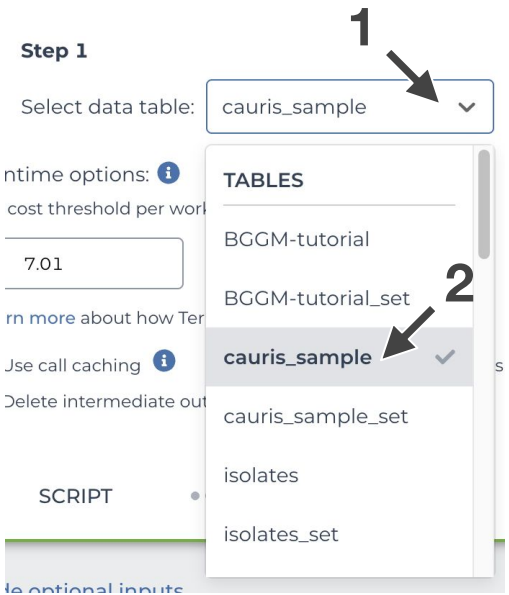
1. Select the **cauris_sample** data table from the dropdown menu

2. Fill in the following input parameters:

Task name	Variable	Input value
theiaeuk_illumina_pe	read1	this.raw_reads1
theiaeuk_illumina_pe	read2	this.raw_reads2
theiaeuk_illumina_pe	samplename	this.cauris_sample_id

References the selected table

3. Click **Save**



Submit the workflow

The easiest way to submit a workflow is through the Terra GUI

1. Click **Select Data**
2. Select all the samples in the **cauris_sample** table
3. Click **OK**
4. ~~Click **Launch**~~ Please do not do this last step, as we will submit the workflow in R

The GUI makes it difficult to subset data. For more complex data manipulation, you may want to submit a workflow through R, using an interactive session

Submit the workflow

1. Open RStudio in your interactive session

2. Create an entity set with the samples for which you want to run the workflow:

```
# Create an entity set containing all the samples we want to run the workflow for
cauris_set_id <- "GEMSTONE_Terra_workshop"  # Define your custom set name
sample_ids <- cauris$cauris_sample_id

# This dataframe must have two specific columns: the set ID and the targeted entity ID
cauris_set <- data.frame(
  membership_id = rep(cauris_set_id, length(sample_ids)),
  cauris_sample = sample_ids
)

# Format the table names for Terra's data model ('membership:' and '_id' suffixes)
colnames(cauris_set) <- c(paste0("membership:", "cauris_sample_set_id"), "cauris_sample")

# Upload to Terra
avtable_import_set(.data = cauris_set, set = "membership:cauris_sample_set_id",
     member = "cauris_sample", origin = "cauris_sample", namespace = project,
     name = workspace)
```

Submit the workflow

3. Define the workflow inputs. You can also define them in the GUI and click **Save**

```
# Define the name of the workflow to run
workflow_name <- "FungalQC"

# Pull existing inputs into an R environment variable
wf_config <- avworkflow_configuration_get(project, workflow_name, project, workspace)

# Define inputs
wf_config$inputs[["theiaeuk_illumina_pe.read1"]] <- "this.raw_reads1"
wf_config$inputs[["theiaeuk_illumina_pe.read2"]] <- "this.raw_reads2"
wf_config$inputs[["theiaeuk_illumina_pe.samplename"]] <- "this.cauris_sample_id"

# Change the configuration settings to point to your new set entity type
wf_config$rootEntityType <- "cauris_sample_set"

# Push changes to Terra
fapi$update_workspace_config(project, workspace, project, workflow_name, wf_config)
```

9 Submit the workflow

4. Submit the workflow

```
# Submit workflows
submission <- AnVIL::Terra()$createSubmission(
  workspaceNamespace = project,
  workspaceName = utils::URLencode(workspace),
  methodConfigurationNamespace = project,
  methodConfigurationName = wf_config$name,
  entityType = "cauris_sample_set",
  entityName = cauris_set_id,
  useCallCache = TRUE,
  expression = "this.cauris_samples"
)

if (submission$status_code == 201) {
  print("Job submitted.")
} else {
  print("Submission failed: ")
  print(submission)
}
```

Uses cached results, if available, instead of computing them again

Runs the workflow for samples in the **cauris_sample** table assigned to the **cauris_set_id** in the **cauris_sample_set** table

Count the number of isolates per clade

```
# Reload the cauris table so it includes the workflow outputs
cauris <- avtable("cauris_sample")

# Count the number of isolates per clade and NH
clade_counts <- cauris %>%
  group_by(nh_code, cladetyper_clade) %>%
  summarise(num_isolates = length(read1_clean), .groups = 'drop') %>%
  pivot_wider(names_from = nh_code, values_from = num_isolates,
values_fill = 0)

print("Number of clade types per nursing home:")
print(clade_counts)
```

	CARD	STOP
Clade1	2	4
Clade3	3	1

11 Generate a phylogenetic tree for clade I

We will use the FungalTree workflow to generate a phylogenetic tree for clade I isolates

The FungalTree workflow takes a set of samples as input

Requires an entity set table, instead of a regular data table

We can use *this.cauris_samples.<colname>* to use a column in the **cauris_sample** table as input to a workflow

this.cauris_sample**s**.column_name

References the entity set table

Tells Terra to look for the column in the cauris_sample table

Clade1 = {gem00001, gem00002}
Clade2 = {gem00003}

membership:cauris_sample_set_id	cauris_sample
Clade1	gem00001
Clade1	gem00002
Clade3	gem00003

Points to sample *gem00003* in the **cauris_sample** table

Generate a phylogenetic tree for clade I

1. Create the **cauris_sample_set** entity set table in R:

```
for (clade in unique(cauris$cladetyper_clade)) {

  # Subset sample IDs from this clade
  clade_ids <- (cauris %>% filter(cladetyper_clade == clade))$cauris_sample_id
  clade_set <- data.frame(
    membership_id = rep(clade, length(clade_ids)),
    cauris_sample = clade_ids
  )

  # Format the table names for Terra's data model ('membership:' and '_id' suffixes)
  colnames(clade_set) <- c(paste0("membership:", "cauris_sample_set_id"), "cauris_sample")

  # Upload to Terra
  avtable_import_set(.data = clade_set, set = "membership:cauris_sample_set_id",
    member = "cauris_sample", origin = "cauris_sample", namespace = project,
    name = workspace
  )

}
```

11 Generate a phylogenetic tree for clade I

2. Import the broadinstitute/idmp-fungal-pipelines/FungalTree workflow from Dockstore

3. Select **Version:** main

4. Select the **cauris_sample_set** data table

5. Click **Select Data**, and select Clade1 samples

6. Define the workflow inputs and outputs:

11 Generate a phylogenetic tree for clade I

Task name	Variable	Input value
FungalTree	analysis_name	this.cauris_sample_set_id
FungalTree	indel_filter_expr	"QD < 2.0 || FS > 60.0 || MQ < 40.0"
FungalTree	input_bams	this.cauris_samples.filtered_bam
FungalTree	input_samples	this.cauris_samples.cauris_sample_id
FungalTree	ref_gbff	"gs://theiagen-public-files/terra/candida_auris_refs/Cauris_Clade1_GCA_002759435_Cauris_B8441_V2_genomic.gbff"
FungalTree	snp_filter_expr	"QD < 2.0 || FS > 60.0 || MQ < 40.0"

Generate a phylogenetic tree for clade I

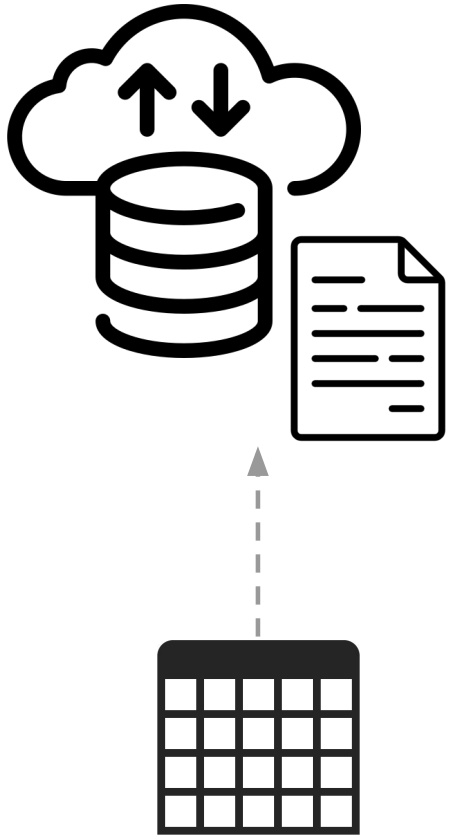
Task name	Variable	Output value
FungalTree	hard_filtered_gvcf	this.hard_filtered_gvcf
FungalTree	IqTree2_ml_tree	this.iqtree2_ml_tree
FungalTree	IqTree2_model_used	this.iqtree2_model_used
FungalTree	IqTree2_run_date	this.iqtree2_run_date

12 Plot the phylogenetic tree

The phylogenetic tree, in Newick format, was written to the *iqtree2_ml_tree* column in the **cauris_sample_set** table

The column has a path to the file, not the tree itself

We must read the file from its Google Bucket



Plot the phylogenetic tree

```
# Load the entity set table with the FungalTree outputs
cauris_set <- avtable("cauris_sample_set")

clade <- "Clade1"
newick_col <- "iqtree2_ml_tree" # Column with Newick tree

# Read Newick file from the Google Bucket
newick <- gcs_get_object(
  (cauris_set %>%
    filter(cauris_sample_set_id == clade)) %>%
    pull(newick_col),
  bucket = bucket
)
# Read the first line (the Newick tree) from the stream
newick  <- readLines(textConnection(rawToChar(newick)))[1]

# Use ape to plot the tree
tree <- read.tree(text=newick)
```

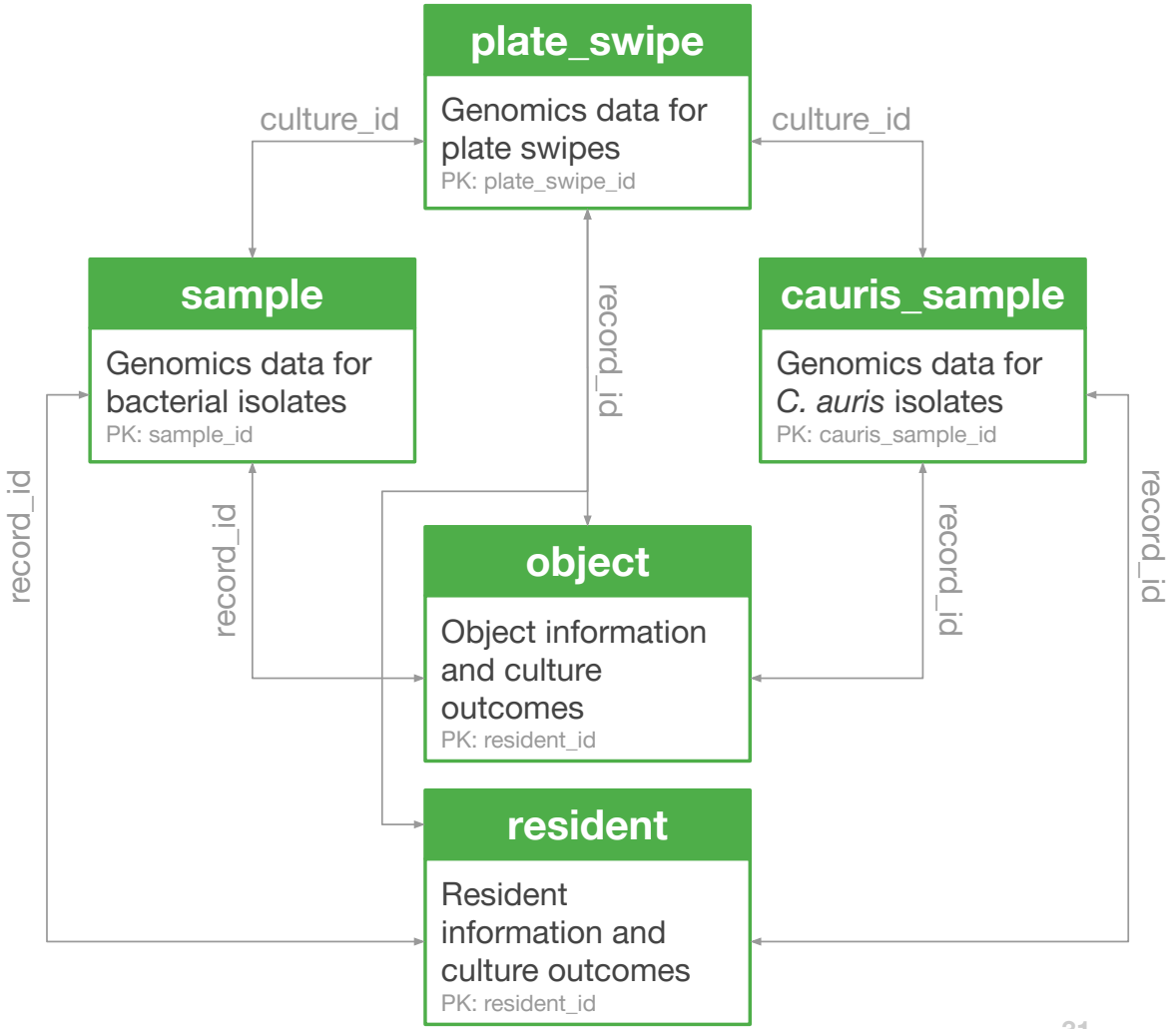
Join the *C. auris* and resident tables

We will now search for resident characteristics associated with *C. auris* clade I

Using retrospective data

Upload data/cauris_sample.retrospective.tsv to your workspace

Need to fetch resident metadata and map it to our isolates



Join the *C. auris* and resident tables

```
# Read the retrospective C. auris table
cauris_retro <- avtable("cauris_sample_retrospective")

data <- left_join(cauris_retro, resident,
  suffix = c("_cauris", ""),
  by = c("record_id" = "resident_id")
)

# Remove columns with all NaNs
data <- data[ , colSums(is.na(data)) < nrow(data)]

# Remove samples without clade assignments; keep samples with
# status == preferred, and remove samples without a unique person ID
data <- data %>%
  drop_na(cladetyper_clade) %>%
  filter(status == "preferred") %>%
  drop_na(person_id)
```

Try your own analyses

Can you detect any resident characteristics that are associated with clade I colonization?

Column	Description
person_id	Unique ID for each sampled resident. Useful to link record IDs that represent the same person across time
age	Age of the resident
swab_date	Sampling date in days since a shrouded date
nh_code	Nursing home
day_of_stay	Length of stay in the nursing home, in days
vSNF_bed	Whether the resident was on a ventilator-equipped bed
wounds	Whether the resident had wounds at time of sampling
indwelling_device	Whether the resident had an indwelling medical device at time of sampling

Share your findings!

Creating your own Terra workflow

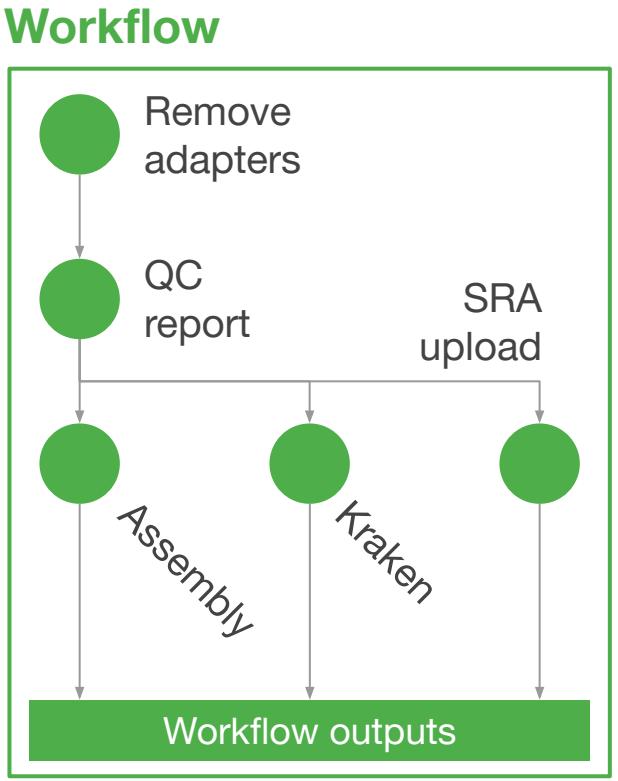
Terra workflows are written in Workflow Description Language (WDL)

Tasks linked together through inputs and outputs

Each task runs in its own VM, in parallel for multiple samples

Tasks run on Docker containers

- Avoids having to install tools in VMs
- Avoids requirements conflicts
- Easy to reuse public containers



WDL task structure

```
version 1.0

task bakta {
  input {
    File assembly
    File bakta_db
    String samplename
    Int cpu = 8
    Int memory = 16
    String docker
    Int disk_size = 100
    Boolean proteins = false
    Boolean compliant = false
    File? prodigal_tf
    String? bakta_opts
  }
  command <<<
  date | tee DATE
  bakta --version | tee BAKTA_VERSION

  (...)
```

The input block defines the task inputs

Input variables can be used in the command and output blocks

WDL task structure

```
command <<<
date | tee DATE
bakta --version | tee BAKTA_VERSION

# Extract Bakta DB
mkdir db
time tar xzvf ~{bakta_db} --strip-components=1 -C ./db

# Install amrfinderplus db
amrfinder_update --database db/amrfinderplus-db
amrfinder --database_version | tee AMRFINDER_DATABASE_VERSION

bakta ~{bakta_opts} --db db/ --threads ~{cpu} \
  --prefix ~{samplename} --output ~{samplename} \
  ~{true='--compliant' false='' compliant} \
  ~{true='--proteins' false='' proteins} \
  ~{'--prodigal-tf ' + prodigal_tf} \
  ~{assembly}

>>>
output {
  (...)
```

The command block contains commands run in the VM

All task outputs should be written to files

WDL task structure

```
  output {
    File bakta_tsv = "~{samplename}/~{samplename}.tsv"
    File bakta_txt = "~{samplename}/~{samplename}.txt"
    String bakta_version = read_string("BAKTA_VERSION")
  }
  runtime {
    memory: "~{memory} GB"
    cpu: cpu
    docker: docker
    disks:  "local-disk " + disk_size + " SSD"
    disk: disk_size + " GB"
    maxRetries: 3
  }
}
```

The output block defines the task outputs

- All other files generated in the VM are deleted
- Can be used by downstream tasks

The runtime block specifies VM characteristics

- Number of cores
- Memory
- Docker image

WDL workflow structure

You can find an example of a WDL workflow here

GEMSTONE Isolates Workflow

Making a workflow available in Terra

1. Upload WDL files to a public GitHub repository
2. Create a .dockstore.yml file
3. Create a Dockstore account and link it with your GitHub account
4. Publish your workflow in Dockstore
5. Import the workflow into Terra

More resources:

- WDL documentation
- Getting started with WDL
- Terra WDL course

When should you use a workflow?

Workflow	Interactive session
Frequent submissions	One-time analyses
Requires multiple tools	Single tool
Set and forget, with minimal changes over time	Frequent changes
Well-defined protocols and pipelines	Testing and experimenting

Further reading

GEMSTONE Terra Documentation

GEMSTONE Workflows Documentation

Terra Documentation

Reach out with questions!
mcarvalh@broadinstitute.org